

EXPERIMENTAÇÃO DE SOFTWARE · PUCMINAS · 01/2026

# GraphQL vs REST

## Um Experimento Controlado

---

GitHub REST API v3 vs GraphQL API v4

100 repositórios · 1.000 medições · 0 falhas/timeouts

AUTORES

**Augusto Fuscaldi e Filipe Faria**

# Contexto e Problema

- **GraphQL:** linguagem de consulta em grafos (Facebook); o cliente especifica exatamente quais campos quer em uma única requisição.
- **REST:** cada endpoint retorna uma representação fixa, com frequência trazendo dados a mais (*over-fetching*) ou exigindo várias chamadas (*under-fetching*).
- Não há consenso claro sobre o ganho real de desempenho ao adotar GraphQL.
- **Ambiente controlado:** a API do GitHub expõe os mesmos dados via REST v3 e GraphQL v4, eliminando diferenças de infraestrutura, backend e modelagem.
- **Tarefa idêntica** nos dois tratamentos: obter os metadados de um repositório.

# Questões de Pesquisa

## RQ1

Respostas às consultas **GraphQL** são mais **rápidas** que as respostas REST?

**Métrica: tempo (ms)**

## RQ2

Respostas às consultas **GraphQL** têm **tamanho menor** que as respostas REST?

**Métrica: tamanho (bytes)**

Teste pareado por repositório sobre a **mediana das diferenças**; hipóteses bidirecionais (duas pontas) por rigor estatístico.

# Hipóteses

## RQ1 · Tempo

**H0:** mediana das diferenças pareadas (REST - GraphQL) = 0

**H1:** tempo do GraphQL difere do tempo do REST (mediana  $\neq$  0)

## RQ2 · Tamanho

**H0:** não há diferença de tamanho entre REST e GraphQL

**H1:** tamanho do GraphQL difere do tamanho do REST

Expectativa teórica: a consulta GraphQL pede apenas um subconjunto de campos, enquanto o REST retorna o recurso completo; espera-se GraphQL com respostas **menores** (RQ2) e possivelmente mais **rápidas** (RQ1).

# Desenho do Experimento

- **Fator único:** tipo de API, com dois níveis (REST e GraphQL).
- **Variável de bloco:** o repositório consultado; cada repositório recebe os dois tratamentos, o que permite um teste pareado.
- **Medidas repetidas** (within-subject) com blocagem por repositório e ordem de aplicação aleatorizada (randomized complete block design).
- **K = 5 repetições** por tratamento; a sequência das 1.000 chamadas é embaralhada globalmente para distribuir variações de carga, cache e rede.
- Uma requisição de **aquecimento** (uma REST e uma GraphQL) é descartada para neutralizar o custo de conexão e TLS inicial.

# Variáveis e Tratamentos

## Variáveis dependentes

- Tempo de resposta em ms (medido com `time.perf_counter`, incluindo rede e handshake).
- Tamanho do corpo da resposta HTTP, em bytes.

## Tratamentos

### T1 · REST

#### GET

**/repos/{owner}/{repo}**

Recurso completo, sem seleção de campos (mais de 100 campos).

### T2 · GraphQL

#### POST /graphql

Query com apenas os 10 campos equivalentes aos usados a partir da resposta REST.

# Medições e Unidade de Análise

- **N = 100** repositórios x **K = 5** repetições x **2** tratamentos = **1.000 medições brutas** (500 por tratamento).
- Para evitar **pseudo-replicação**, cada repositório é resumido pela **mediana das suas 5 repetições** antes do teste estatístico.
- Isso gera **100 pares (REST, GraphQL)** independentes, um por repositório, que alimentam os testes pareados de RQ1 e RQ2.
- Amostra: repositórios públicos com mais de **1.000 estrelas** (Search API), cobrindo várias linguagens, tamanhos de descrição e presença ou não de licença.
- As 1.000 medições brutas são preservadas para os gráficos descritivos de variabilidade.

# Ameaças à Validade

## Interna

Latência de rede, carga do servidor e cache de CDN podem afetar o tempo. Mitigado por aleatorização global, K=5 e mediana

## Externa

Resultados específicos do GitHub e de uma leitura simples; não generalizam para escrita (mutations) ou consultas profundas.

## Constructo

A API REST do GitHub não permite seleção de campos; a diferença de tamanho mede também o over-fetching estrutural dessa API.

## Conclusão

A escolha entre teste t pareado e Wilcoxon é feita por Shapiro-Wilk sobre as diferenças pareadas, sem assumir normalidade indevidamente.



# Execução do Experimento

Coleta real em **26/06/2026, das 15:30 às 15:48 UTC** (cerca de 18 min e 50 s), com `run_experiment.py` autenticado por token pessoal

| Item                                 | Resultado                      |
|--------------------------------------|--------------------------------|
| Repositórios (objetos experimentais) | 100 (estrelas > 1.000)         |
| Repetições por tratamento (K)        | 5                              |
| Medições planejadas                  | 1.000 (500 REST + 500 GraphQL) |
| Medições coletadas                   | 1.000                          |
| Respostas com status HTTP 200        | <b>1.000 / 1.000 (100%)</b>    |
| Falhas, timeouts ou rate limit       | <b>0</b>                       |
| Retries necessários                  | <b>0</b>                       |

# Revisão dos Valores Obtidos

Nenhuma exclusão necessária: as 1.000 medições têm ok = True.  
Estatísticas descritivas das medições brutas (500 por tratamento):

| Métrica         | Trat.   | Mínimo | Mediana        | Média   | Máximo  | Desvio padrão |
|-----------------|---------|--------|----------------|---------|---------|---------------|
| Tempo (ms)      | REST    | 593,6  | <b>766,2</b>   | 767,2   | 2.953,0 | 133,4         |
| Tempo (ms)      | GraphQL | 618,4  | <b>791,5</b>   | 790,9   | 1.130,6 | 84,1          |
| Tamanho (bytes) | REST    | 5.057  | <b>6.321,0</b> | 6.265,3 | 7.697   | 618,2         |
| Tamanho (bytes) | GraphQL | 366    | <b>431,5</b>   | 447,5   | 718     | 59,8          |

- O máximo do REST (2.953 ms) é um pico isolado de rede, mantido porque a mediana por repositório já é robusta a esse tipo de outlier.
- O tamanho do GraphQL varia pouco (dp 59,8) porque a query sempre retorna o mesmo conjunto fixo de 10 campos.

# RQ1: Tempo de Resposta

REST · mediana

**764,92 ms**

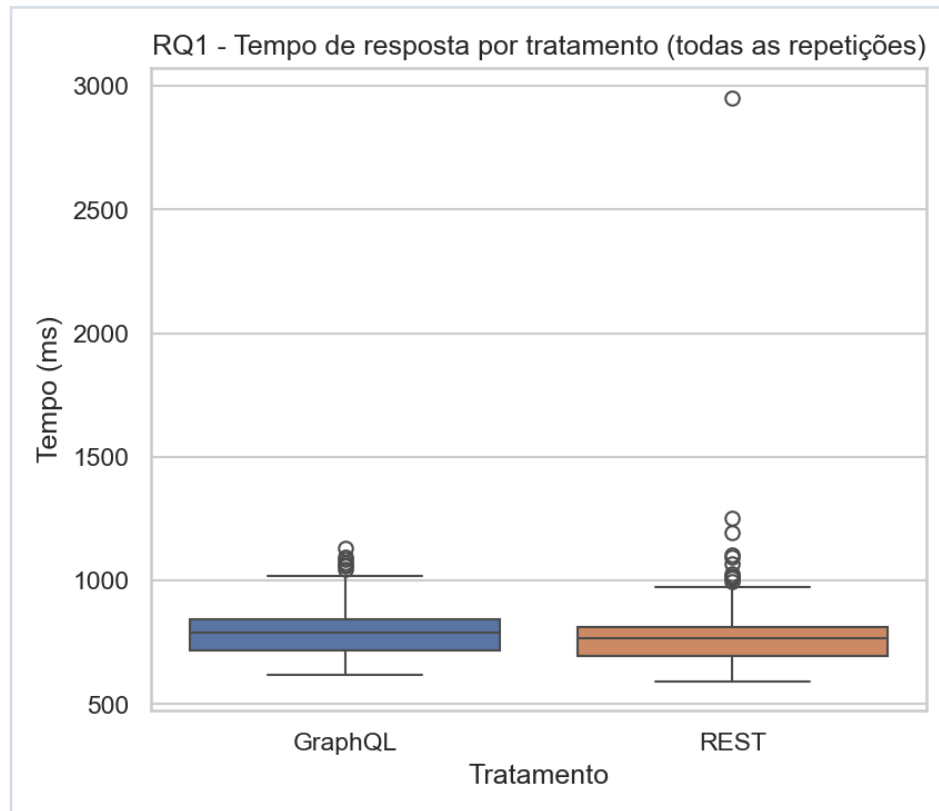
GraphQL · mediana

**794,44 ms**

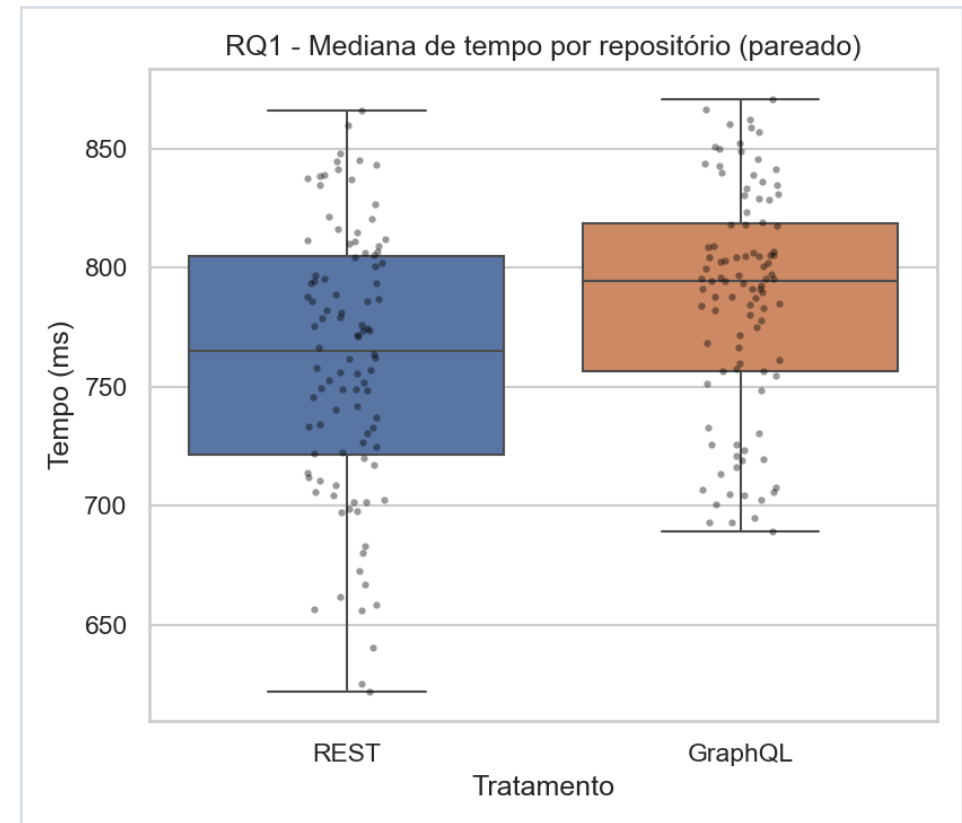
- GraphQL ficou **3,9% mais lento** (cerca de 30 ms) na mediana pareada.
- Shapiro-Wilk na diferença pareada:  $p = 0,8888$  ( $> 0,05$ ), compatível com normalidade.
- Teste escolhido: **t pareado**; estatística = -3,4291;  **$p = 0,00088$** .

**$H_0$  rejeitada ( $p = 0,00088$ ) · diferença pequena**

# RQ1: Distribuição do Tempo

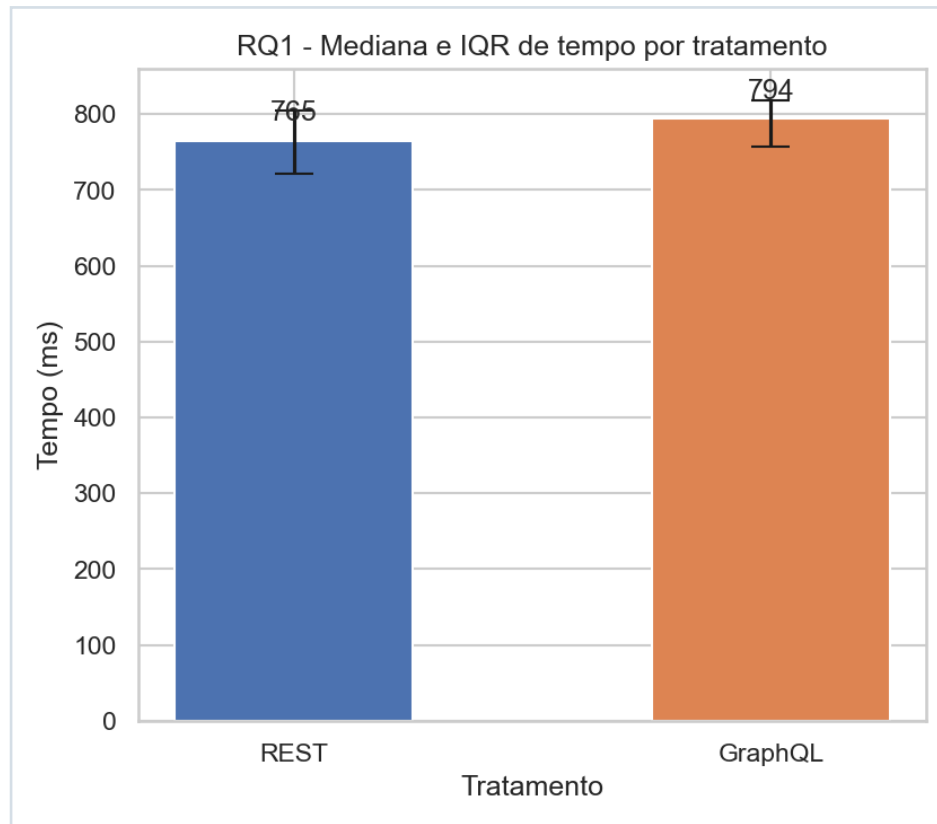


Boxplot bruto (500 medições). As caixas se sobrepõem fortemente: a diferença real fica menos evidente no dado bruto.

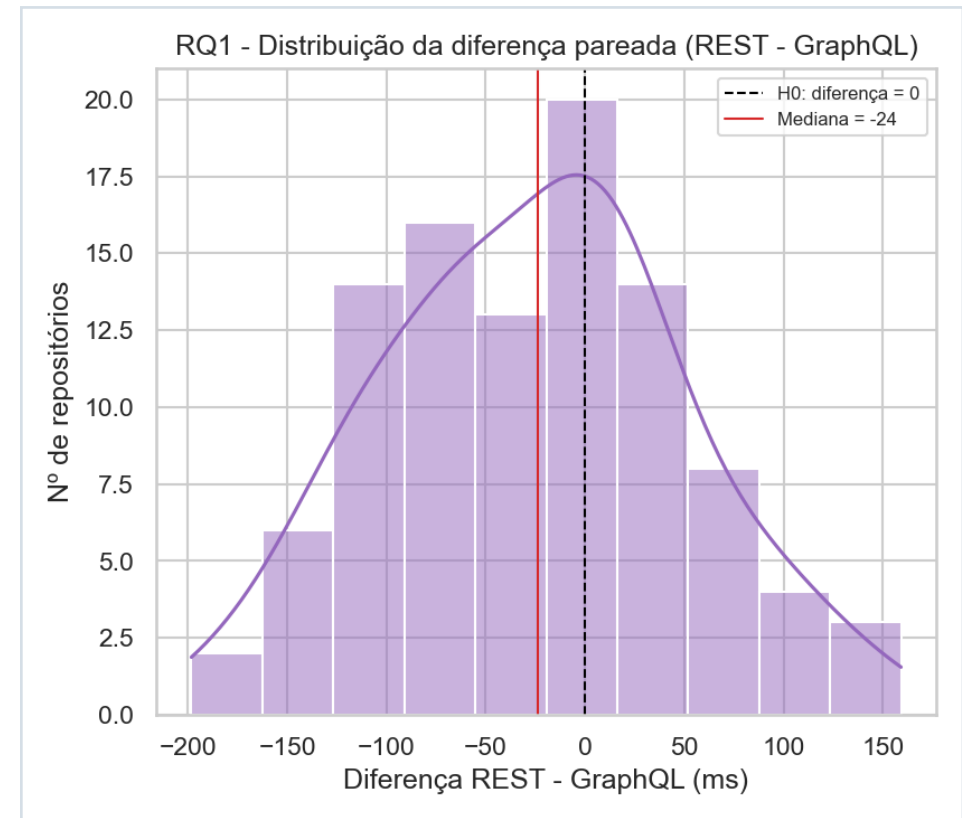


Boxplot pareado por repositório (100 medianas). GraphQL deslocado para cima, com grande sobreposição: efeito real, porém pequeno.

# RQ1: Mediana e Diferença Pareada

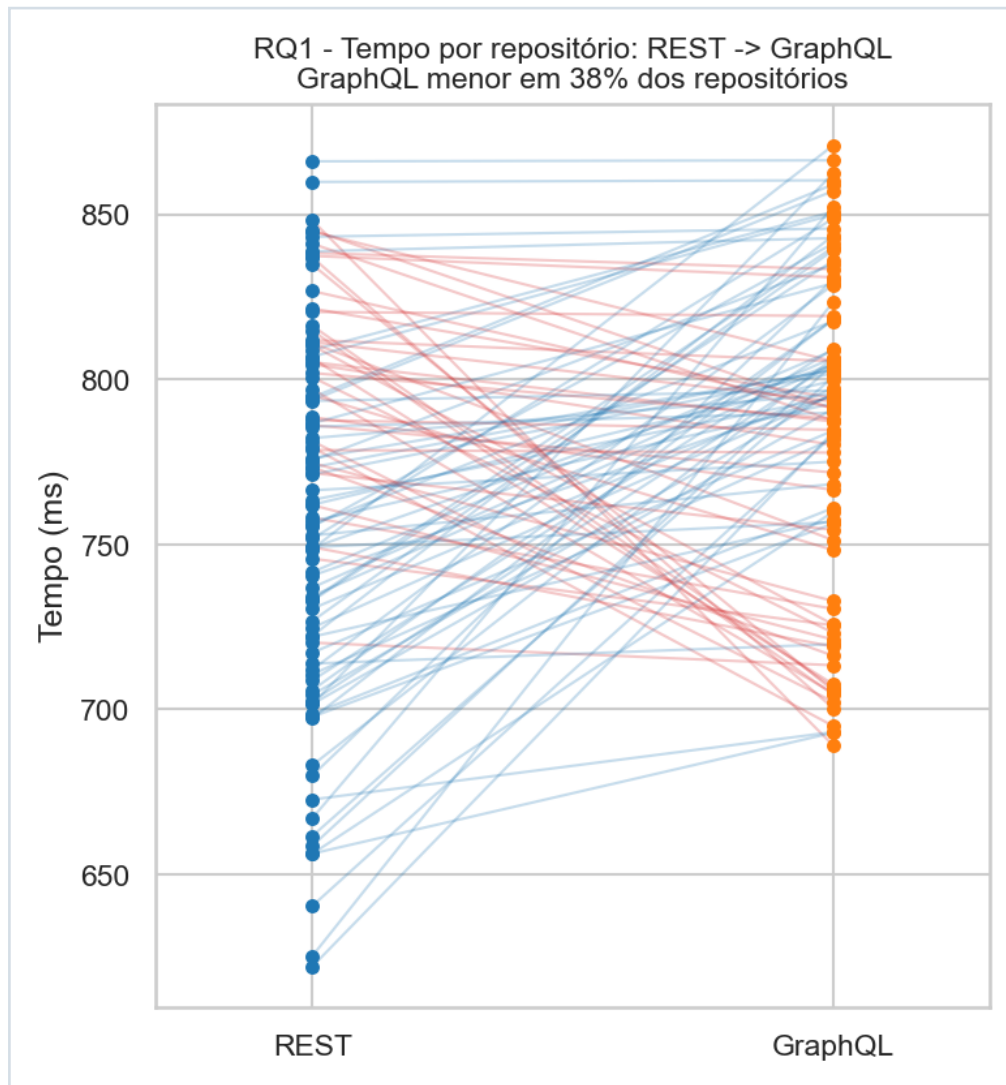


Mediana e IQR por tratamento (765 vs 794 ms). Barras de erro se sobrepõem: diferença pequena frente à dispersão.



Diferença pareada (REST menos GraphQL). Quase simétrica e centrada perto de zero, mediana de -24 ms: efeito pequeno.

# RQ1: Slopegraph por Repositório



## GraphQL mais rápido em apenas 38% dos repositórios

- Cada linha liga o tempo de um repositório no REST ao do GraphQL.
- Vermelho: GraphQL mais rápido ali; azul: mais lento. As linhas se cruzam muito.
- Efeito significativo na mediana, mas **não consistente** caso a caso (o REST venceu em 62%).

## RQ2: Tamanho da Resposta

REST · mediana

**6.321,0 bytes**

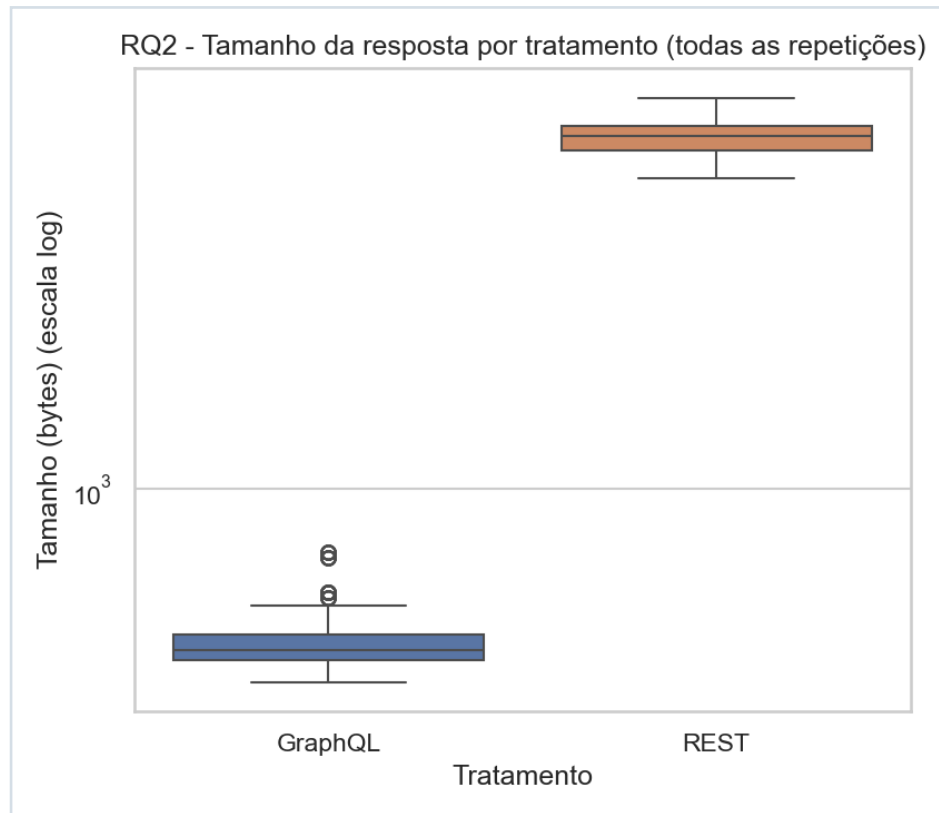
GraphQL · mediana

**431,5 bytes**

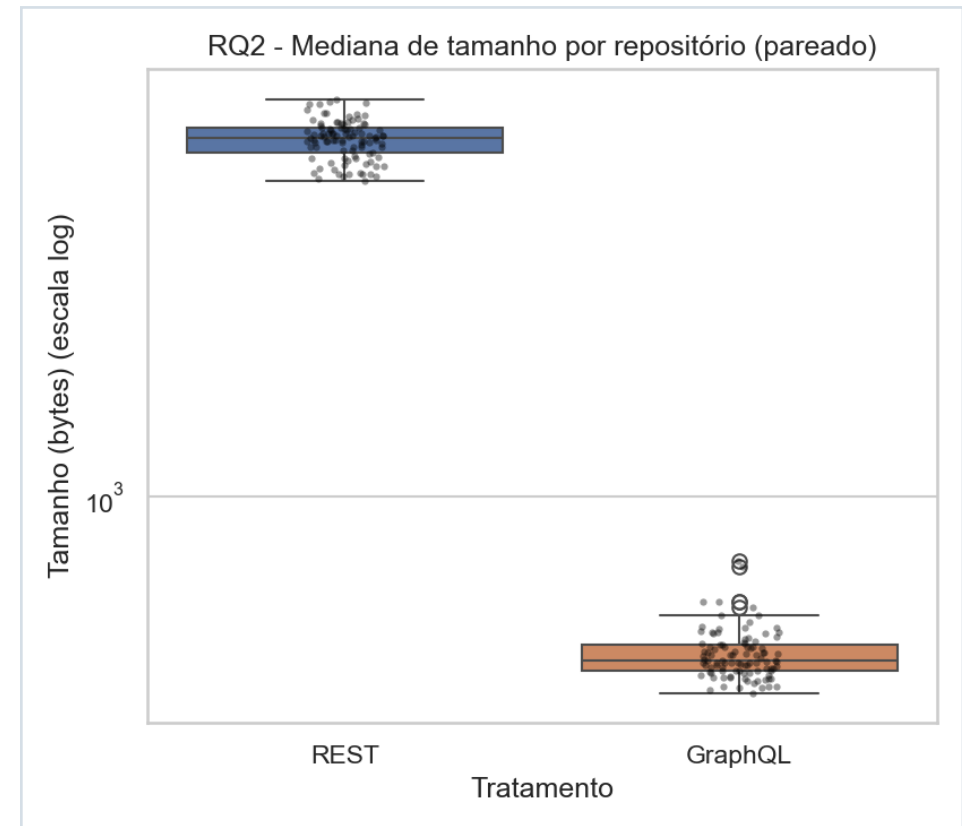
- GraphQL **93,2% menor** (cerca de 14,6 vezes menos dados) na mediana pareada.
- Shapiro-Wilk na diferença pareada:  $p = 0,04348$  ( $< 0,05$ ), diferenças não normais.
- Teste: **Wilcoxon signed-rank**;  $W = 0$ ;  $p \sim 0$  ( $3,9 \times 10^{-18}$ ).

**H0 rejeitada ( $W = 0$ ) · GraphQL menor em todos os 100 repositórios**

## RQ2: Distribuição do Tamanho (escala log)



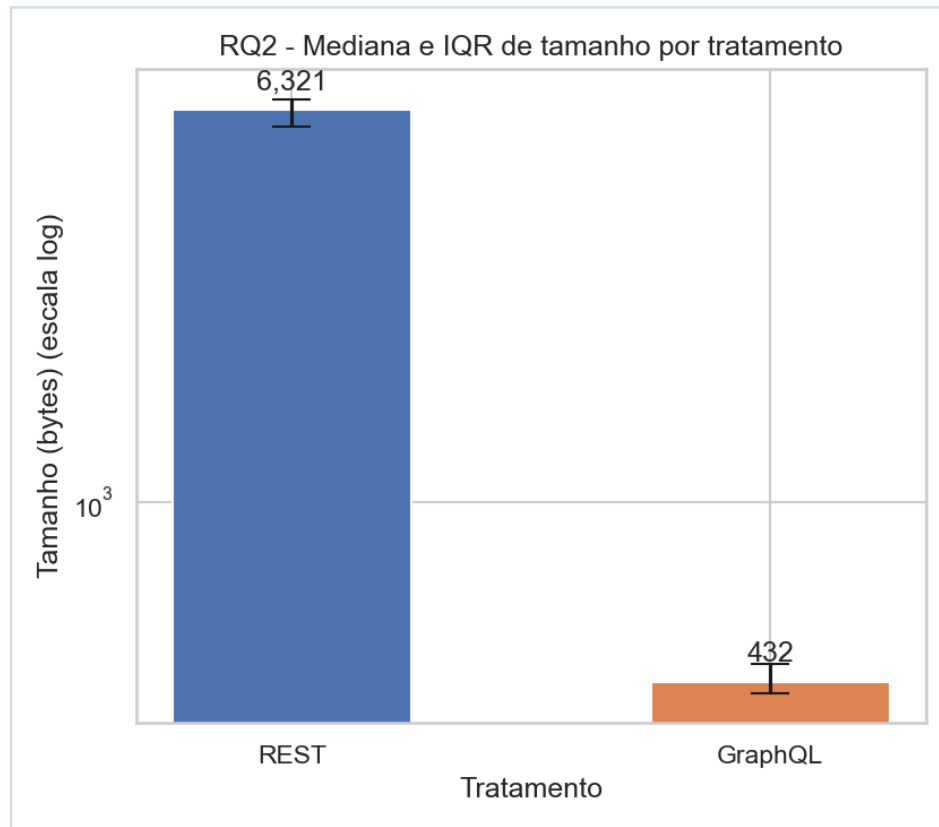
Boxplot bruto, eixo Y em escala log. As caixas REST e GraphQL ficam totalmente separadas, sem qualquer sobreposição.



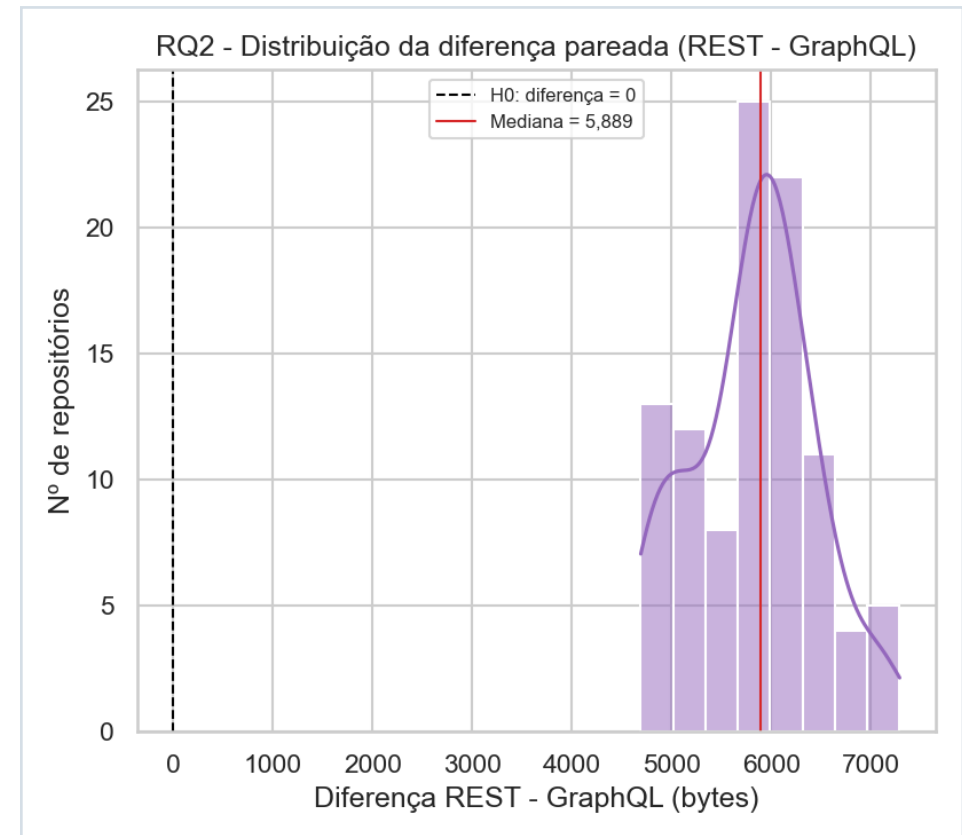
Boxplot pareado por repositório. A separação total entre as caixas já comunica a magnitude do efeito antes de qualquer teste.



# RQ2: Mediana e Diferença Pareada

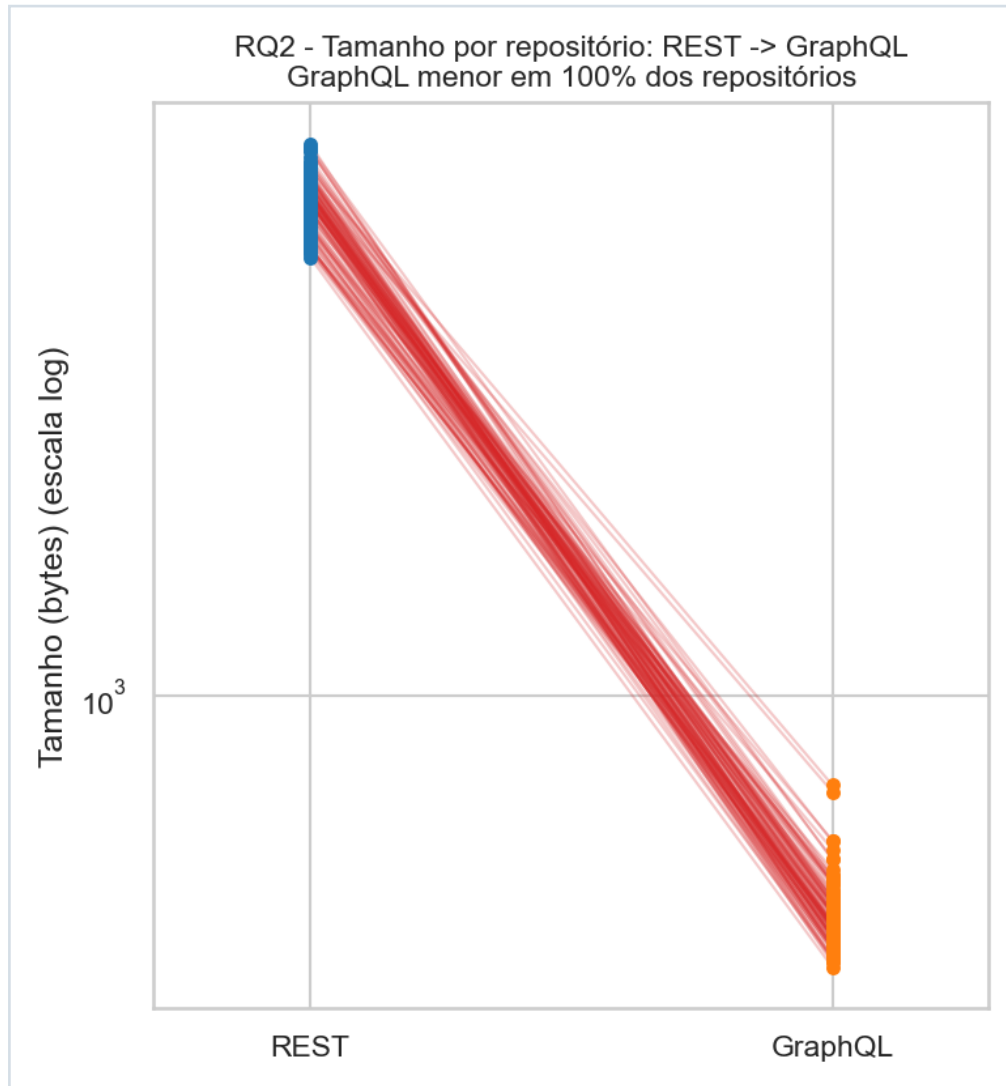


Mediana e IQR por tratamento (6.321 vs 432 bytes) em escala log. A diferença de altura é a forma mais direta do achado.



Diferença pareada inteiramente à direita do zero (entre ~4.600 e ~7.300 bytes). Nenhum repositório com diferença negativa.

# RQ2: Slopegraph por Repositório



## GraphQL menor em 100% dos repositórios

- As 100 linhas são vermelhas (GraphQL menor) e praticamente paralelas.
- Convergem de 5.000 a 7.700 bytes (REST) para 366 a 718 bytes (GraphQL).
- Nenhum cruzamento: efeito universal, que explica o Wilcoxon com  $W = 0$ .

# Síntese Comparativa

| RQ         | Métrica           | REST      | GraphQL   | Diferença                  | Teste     | p-valor | H0?        |
|------------|-------------------|-----------|-----------|----------------------------|-----------|---------|------------|
| <b>RQ1</b> | Tempo (mediana)   | 764,92 ms | 794,44 ms | GraphQL +3,9% (mais lento) | t pareado | 0,00088 | <b>Sim</b> |
| <b>RQ2</b> | Tamanho (mediana) | 6.321,0 B | 431,5 B   | GraphQL -93,2% (menor)     | Wilcoxon  | ~ 0     | <b>Sim</b> |

- O efeito de **RQ2 é consistente em 100%** dos repositórios, sem exceções.
- O efeito de **RQ1 é real, mas presente em só 38%** dos repositórios; a maioria, isoladamente, favoreceu o REST em tempo.

# Discussão e Conclusão

**RQ1 Não.** GraphQL foi cerca de 3,9% mais lento (significativo, mas pequeno: ~24 a 30 ms em respostas de mais de 700 ms, dominadas por rede). O sinal se inverte em 62% dos repositórios: não há vantagem de velocidade caso a caso.

**RQ2 Sim, de forma marcante e universal.** GraphQL 93,2% menor (~14,6x menos dados), com  $W = 0$  e todos os 100 repositórios favorecendo GraphQL. Confirma o ganho concreto de evitar over-fetching.

**Trade-off:** a economia de payload é grande, consistente e robusta, mas não se traduziu em ganho de tempo aqui; pode surgir com payloads maiores, redes lentas ou cenários de under-fetching.

# Dashboard Consolidado

## Dashboard: GraphQL vs REST, Um Experimento Controlado

GitHub REST API v3 vs GraphQL API v4 | 100 repositórios | 1.000 medições (500 REST + 500 GraphQL) | Coleta: 26/06/2026 15:30 a 15:48 UTC

### Objetos experimentais

100 repositórios públicos  
selecionados por número de estrelas  
K = 5 repetições por tratamento

### Medições coletadas

1.000 medições totais  
500 REST + 500 GraphQL  
0 falhas/timeouts

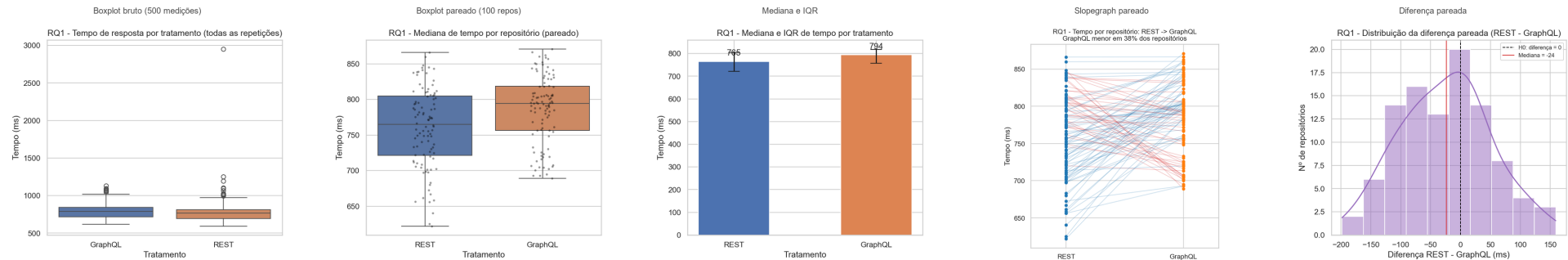
### RQ1: Tempo de resposta

REST: 764,9 ms GraphQL: 794,4 ms  
GraphQL 3.9% mais lento (mediana)  
Teste t pareado,  $p = 0,0088$

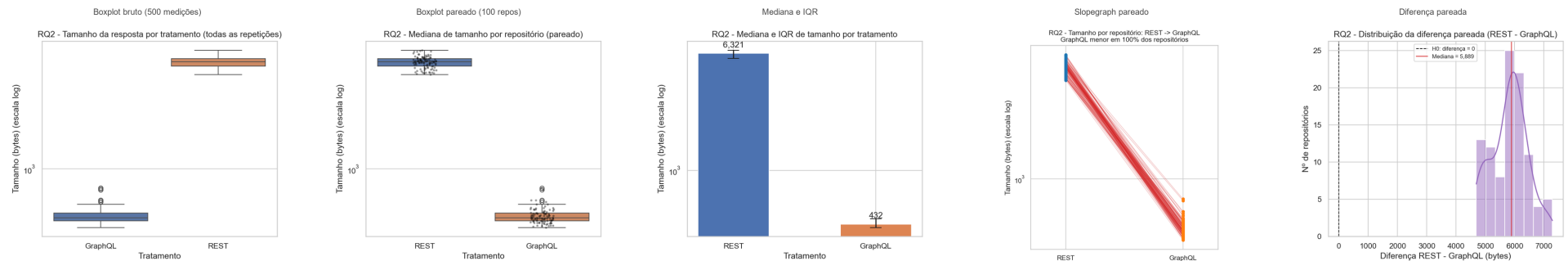
### RQ2: Tamanho da resposta

REST: 6.321,0 bytes GraphQL: 431,5 bytes  
GraphQL 93.2% menor (mediana)  
Wilcoxon signed-rank,  $p < 0,0001$

### RQ1: Respostas GraphQL são mais rápidas que REST?



### RQ2: Respostas GraphQL têm tamanho menor que REST?



Trab-5, Experimentação de Software, PUCMINAS 01/2026. Dados em data/resultados\_brunos.csv, gráficos individuais em imgs/.

Painel único: cabeçalho do experimento, quatro cartões de resumo e duas linhas de gráficos (uma por RQ), permitindo comparar a consistência de RQ1 e RQ2 em uma só tela.